Python simulation of destination-dispatch elevators

# What drives elevator waiting, and what it costs to protect the worst case

Six dispatch algorithms, eight buildings, express layouts, seat rules and guards

**6**
algorithms

**8**
buildings, plus 6 express layouts

**20,880**
simulation runs

**16 million**
simulated trips

**The building sets most of it**
At overload, 60 floors take 42× as long as 10 on the same traffic, and doubling the cars divides the time by 4.2.
Average of the five algorithms of the building study

**Express cars help at the morning rush**
With one car for the top half, `1high31`, average time falls 13% at `up_peak` and rises at the other three patterns.
`marginal_insertion`, heavy traffic

**The guard does the fairness work**
It cuts the longest wait by 47% at heavy traffic for 10% more average time. With it on, seat rules change little.
`marginal_insertion` with `min_time`, on `3car-60floor-6cap`

Yellow means “look here”, never better or worse. Final results. The default guard is on unless a slide says otherwise.

# The model

### Destination dispatch

A passenger enters their floor and destination. The system assigns one car at once, and never changes it.

### The same movement for all

Every car keeps its direction while anyone ahead needs it, and never exceeds its seats. Algorithms only choose the car.

### Stops take no time

One tick is one floor, so a ride always takes its distance. Every difference in time is a difference in waiting.

- **Measured:** average total time per passenger, the assignment's measure, beside the fairness measures: longest wait, p95 wait and times left behind.
- **Results use the default guard unless a slide says otherwise:** a passenger left behind twice is flagged, boards first, and a car keeps a seat for them.
- The software shows results in a fixed order and never ranks them. A person decides what is best.

# Assumptions and simplifications

## Assumptions: the problem as modelled

- One tick is one floor of travel, and time moves a tick at a time.
- Each input row is one trip. The passenger gives their floor and destination when they request.
- Each passenger gets one car at once, and it never changes. Algorithms see only requests already made.
- Every car follows LOOK: it keeps its direction while anyone ahead needs it, and boards only passengers going its way.
- Capacity is never exceeded, so a full car leaves people behind for a later pass.
- Floors run from 1 to n, and all cars start on floor 1. The defaults are 10 floors, 3 cars and 6 seats.
- Wait is the board tick minus the request tick, and total is wait plus ride. “Best” means the lowest average total time, and a person judges it.

## Simplifications: left out on purpose

- Stops take no time: no doors, boarding or acceleration. A ride takes exactly its distance, every difference is waiting, and express gains are a lower bound.
- No transfers or sky lobbies: express cars simply skip floors.
- Idle cars stay where they are, except under `zone_based`, which parks them mid-zone.
- Simulated traffic: 4 patterns, random arrivals over 1,000 ticks, 4 levels from 30% to 120% of a measured limit, and 5 seeds, so compare options seed by seed.
- `marginal_insertion` picks a car by simulating each car forward: under LOOK there’s no position in a route to insert into.
- Ties go to the lowest-numbered car.
- Five planned rules that change how `marginal_insertion` picks a car were left out of scope.

# Six ways to choose a car

| Algorithm | Chooses |
|---|---|
| `round_robin` | The next car in turn |
| `nearest_car` | The best distance score, allowing for direction |
| `zone_based` | The car that owns the request’s block of floors |
| `eta` | The lowest predicted wait for the new passenger |
| `marginal_insertion` | The smallest rise in total time for everyone on the car, found by simulating each car forward |
| `rush_express` | As `marginal_insertion`, except that in a lobby rush the cars split into any-floor, low-rise and high-rise |

Algorithms always appear in this order, which is not a ranking.

# What was run

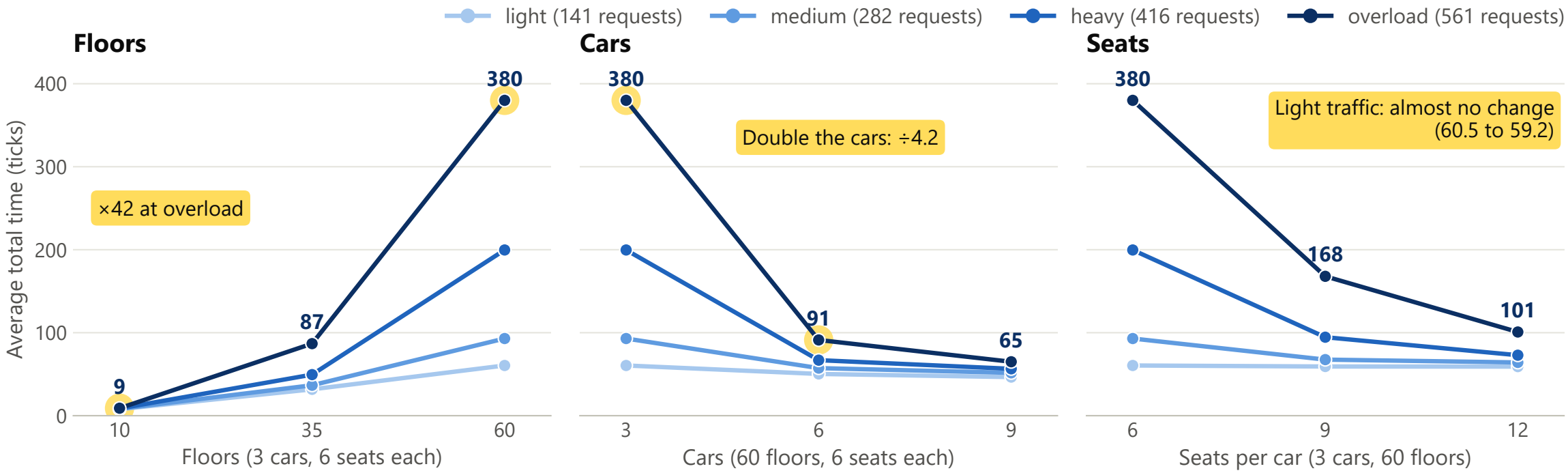
| Study | Buildings | Varied | Runs |
|---|---|---|---|
| Buildings | Eight, from 1 to 9 cars, 10 to 60 floors and 6 to 12 seats | 5 algorithms, at each building's own limit and on one shared traffic | 6,000 |
| Express cars | Five express layouts of `3car-60floor-6cap`, and a 6-car one | 6 algorithms, on the passengers of the building each changes | 4,800 |
| Seat rules | `3car-10floor-6cap`, `3car-60floor-6cap` and an express layout | 6 algorithms × 5 seat settings | 7,200 |
| Guards | The same three | 9 guard settings on `marginal_insertion` | 2,880 |

- **4 traffic patterns:** `uniform`, `up_peak` (most start at the lobby), `down_peak` (most go to it) and `lunch` (both).
- **4 levels:** 30%, 60%, 90% and 120% of each building's measured limit, over 1,000 ticks, on 5 seeds.
- **Every option in a study gets the same passengers,** so a difference comes from the option.

# 1. The building

Eight buildings, changing floors, cars and seats one at a time, every one given the same traffic.

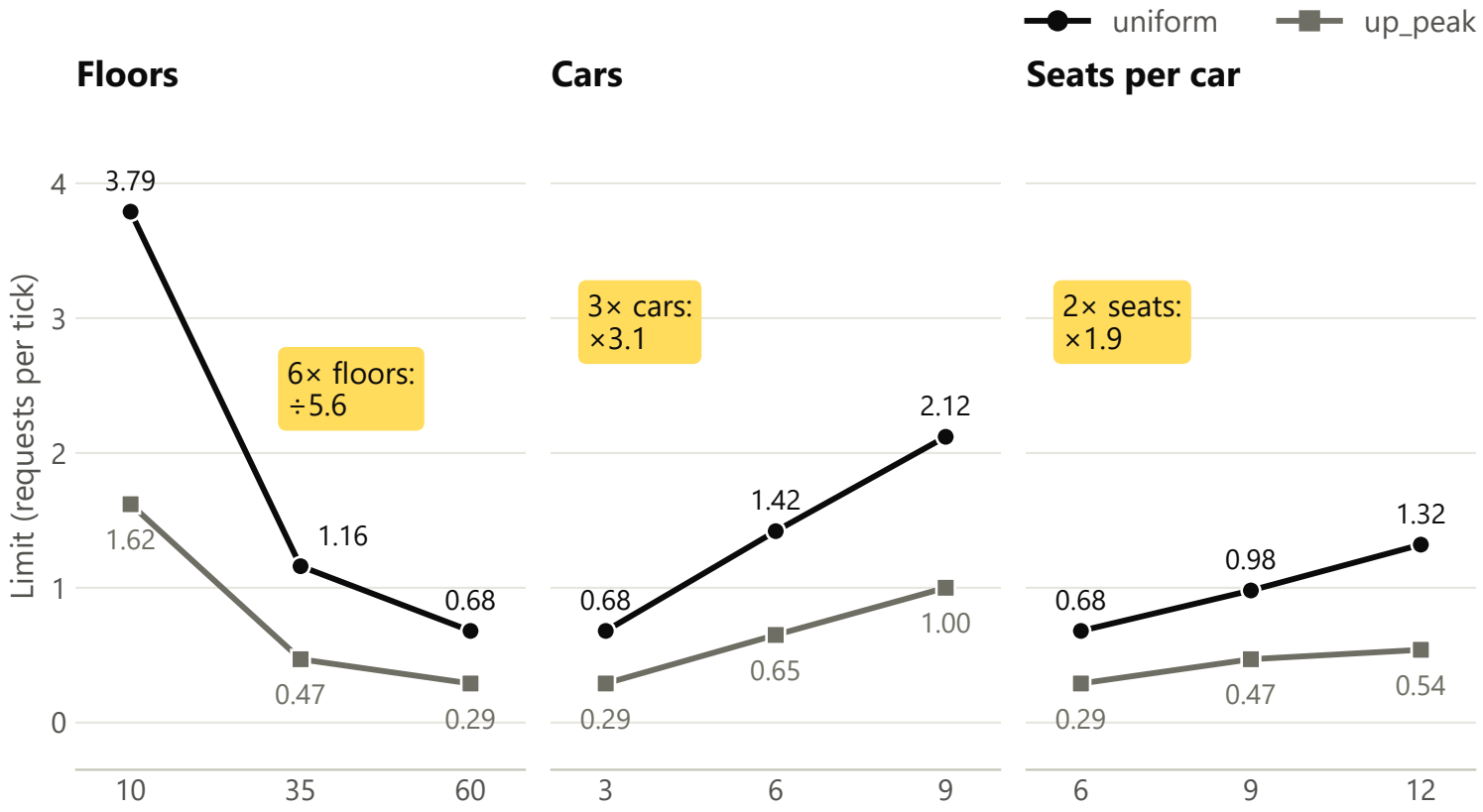
# Floors push time up, and cars and seats pull it down




> **Algorithm:** Averaged over the five algorithms, `round_robin`, `nearest_car`, `zone_based`, `eta` and `marginal_insertion`, each with `min_time` and the default guard, and over the four patterns and five seeds.

- Every building gets the traffic of `3car-60floor-6cap`, so only the building changes.
- At light traffic extra seats change almost nothing, because seats rarely run short. The gaps open as traffic grows.

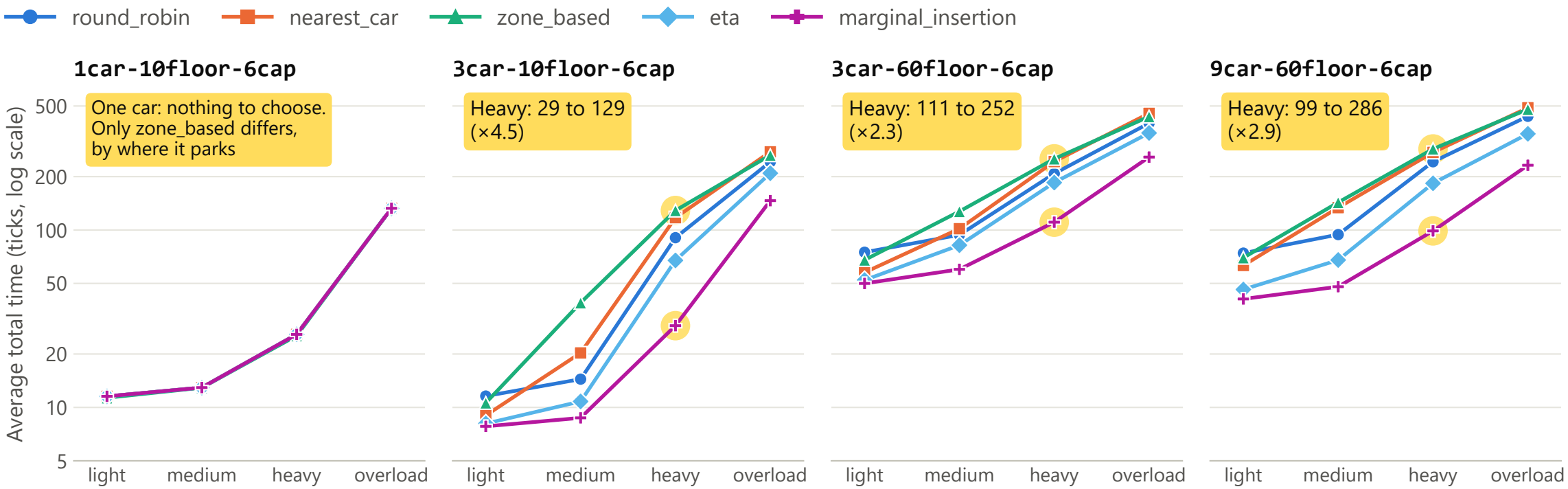
# A building's limit follows its seats per round trip




- The limit is the most traffic a building carries before its queue keeps growing.
- More floors mean longer round trips, so each seat carries fewer people.
- More cars or seats raise it almost in proportion.
- up_peak has the lowest limit on every building with more than one car: everyone leaves from one floor.

**Algorithm:** `marginal_insertion` with `min_time` and no guard, which measured every limit. The limits were then frozen, so every study uses the same passengers.

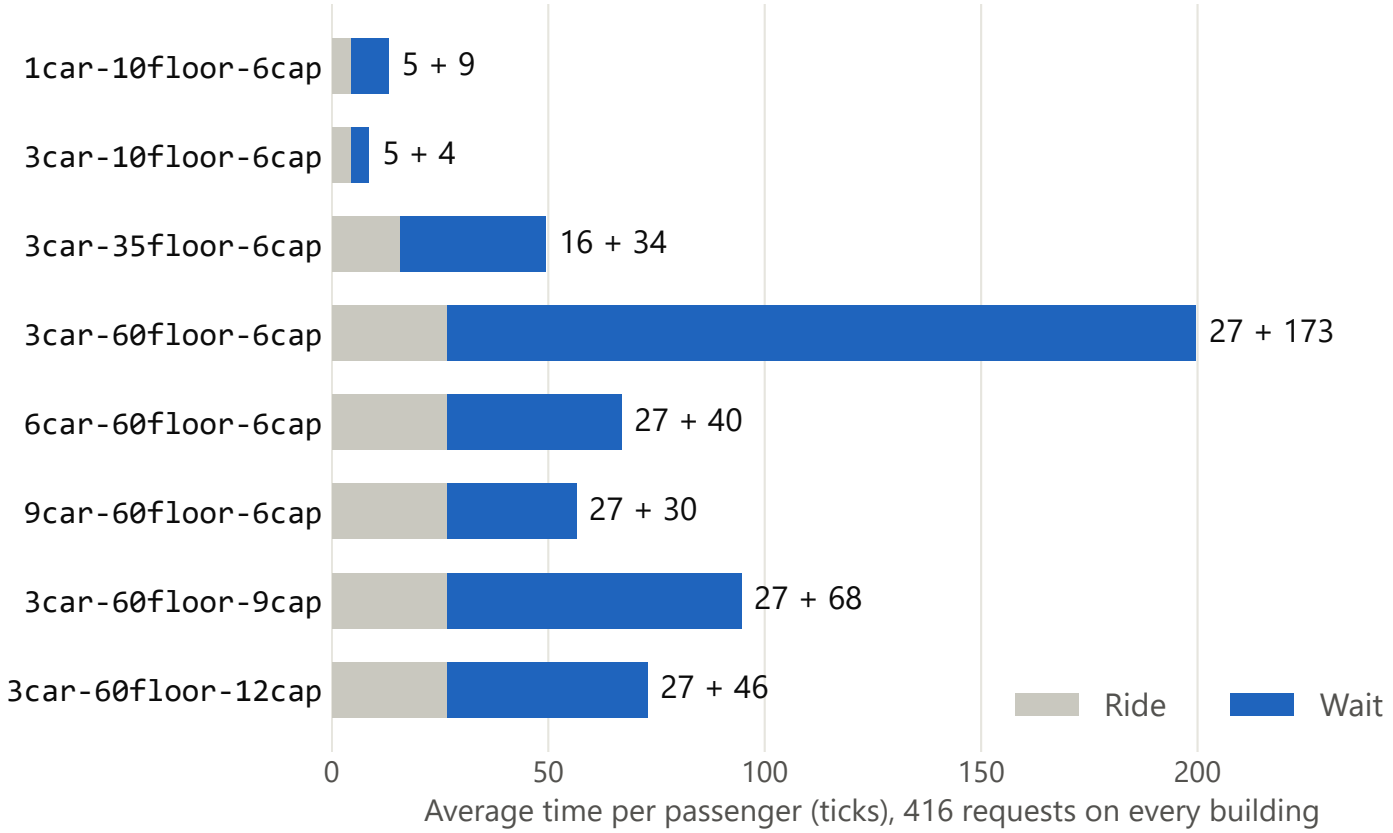
# Car choice matters more as a building gets busy




**Algorithms:** One line for each of the five of the building study, each with `min_time` and the default guard, averaged over the four patterns and five seeds.

- Each building at its own limit, so it is busy in the same proportion. Log scale.
- With one car there is nothing to choose. With three or more, the algorithms spread apart as traffic rises.

# Ride time depends only on height. Everything else is waiting




- Every passenger rides exactly their distance, so the ride part is set by the building's height.
- Cars and seats change only the wait.
- Heavy traffic, on the same arrivals for every building.

> **Algorithm:** Averaged over the five algorithms, `round_robin`, `nearest_car`, `zone_based`, `eta` and `marginal_insertion`, each with `min_time` and the default guard, and over the four patterns and five seeds.

# 2. Express cars

Five layouts of the 3-car, 60-floor reference building, on the same passengers as the building without them.

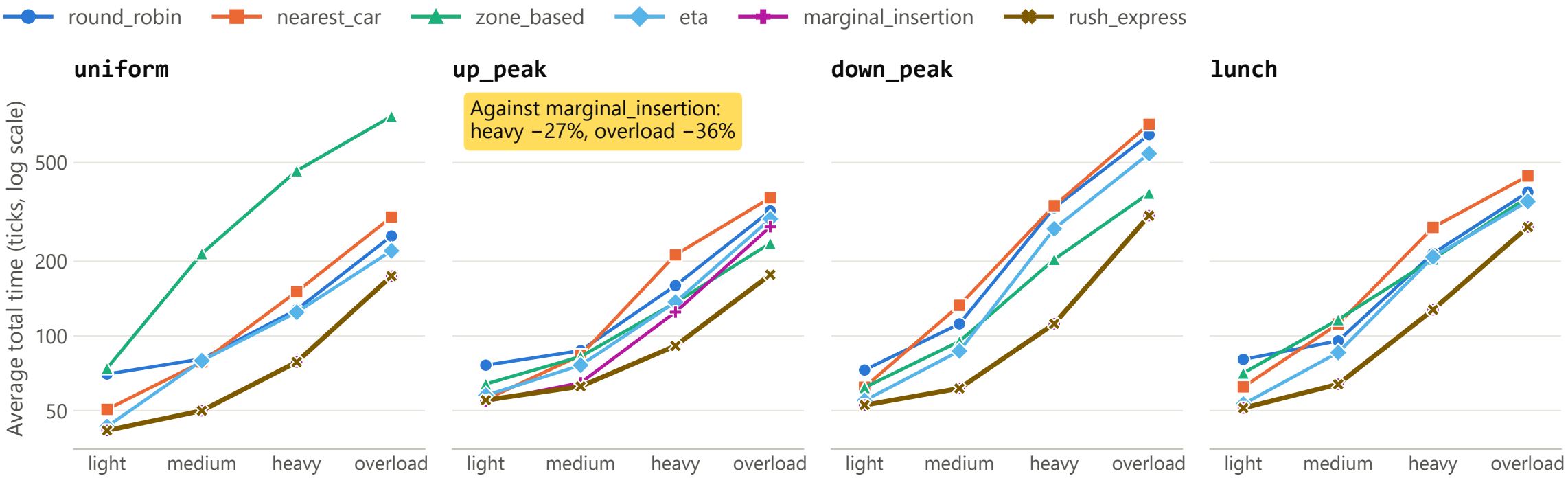
# Express cars help at the morning rush, and mostly cost elsewhere

| Layout, against no express | uniform | up_peak | down_peak | lunch |
|---|---|---|---|---|
| `1high31` | +107% | **−13%** | +8% | +3% |
| `1high41` | +146% | **−10%** | +9% | +9% |
| `2high31` | +756% | **+15%** | +88% | +124% |
| `split30` | +509% | **−30%** | +16% | −4% |
| `overlap` | +60% | **−23%** | +12% | −13% |

> **Algorithm:** `marginal_insertion` with `min_time` and the default guard, at heavy traffic, averaged over five seeds.

- Change in average total time against the same building with no express cars. `1high31` keeps one car for the lobby and floors 31 to 60; `split30` divides the building at floor 30; `overlap` gives two cars overlapping ranges, floors 1 to 40 and the lobby with 21 to 60.
- Stops take no time here, so an express car saves nothing by skipping floors. It helps only by separating traffic, which pays when most trips share the lobby. `split30` and `overlap` also help at `lunch`.

# `rush_express`: run ordinary cars as express, only in a rush



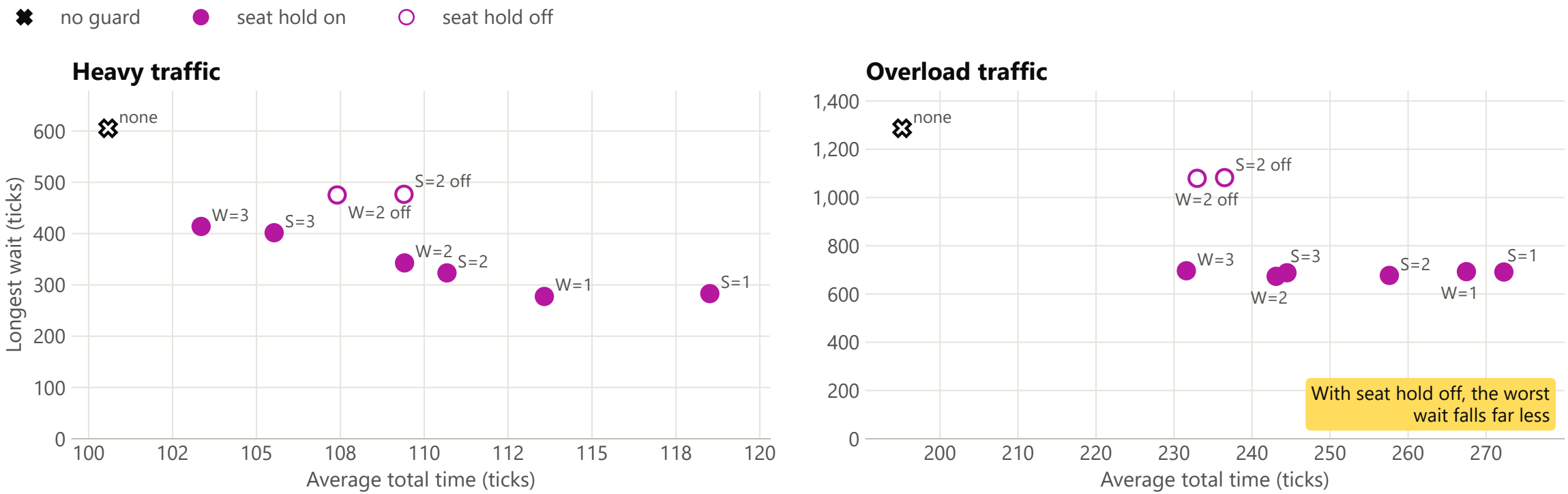

**Algorithms:** One line for each of the six, each with `min_time` and the default guard, on `3car-60floor-6cap` without express cars, averaged over five seeds.

- While 70% of the last 30 requests come from the lobby, the three cars take any-floor, low-rise and high-rise roles. Otherwise it is `marginal_insertion`, so away from `up_peak` its line hides that one.
- Its rule was chosen on seeds 1 to 10 and checked once on seeds 11 to 15.

# 3. Guards

A passenger left behind again and again is flagged, boards first, and has a seat kept for them.
This is the default everywhere else in this deck.

# Tighter limits cost more, and seat hold sets the worst wait




**Algorithm:** `marginal_insertion` with `min_time` on `3car-60floor-6cap`. Each point is one guard setting, averaged over the four patterns and five seeds.

- S: flagged after S skips. W: flagged after W round trips of waiting.
- The default, S = 2 with seat hold, cuts the longest wait by 47% at heavy traffic for 10% more average time.

# Seat hold does most of the work

| | Heavy | | | Overload | | |
|---|---|---|---|---|---|---|
| Guard | Average | Longest wait | Most skips | Average | Longest wait | Most skips |
| No guard | 100.6 | 605 | 6.5 | 195.1 | 1,287 | 14.4 |
| S = 2, seat hold on (default) | 110.7 | 323 | 2.5 | 257.6 | 677 | 6.0 |
| S = 2, seat hold off | 109.4 | **477** | 5.0 | 236.5 | **1,082** | 11.2 |
| W = 2 round trips, seat hold on | 109.4 | 343 | 3.5 | 243.1 | 673 | 6.0 |
| W = 2 round trips, seat hold off | 107.4 | **475** | 5.2 | 233.0 | **1,079** | 11.1 |

> **Algorithm:** `marginal_insertion` with `min_time` on `3car-60floor-6cap`, averaged over the four patterns and five seeds.

- Without seat hold, a flagged passenger still meets a full car, so the longest wait falls much less.
- Most skips is the most times one passenger was left behind, averaged over runs.

# The cost depends on the building

| Default guard against none, `marginal_insertion`, `min_time` | Average total | p95 wait | Longest wait |
|---|---|---|---|
| `3car-60floor-6cap`, heavy | +10% | −3% | −47% |
| `3car-10floor-6cap`, heavy | **+83%** | **+59%** | −38% |
| `3car-60floor-6cap`, overload | +32% | −21% | −47% |
| `3car-10floor-6cap`, overload | +72% | −18% | −55% |

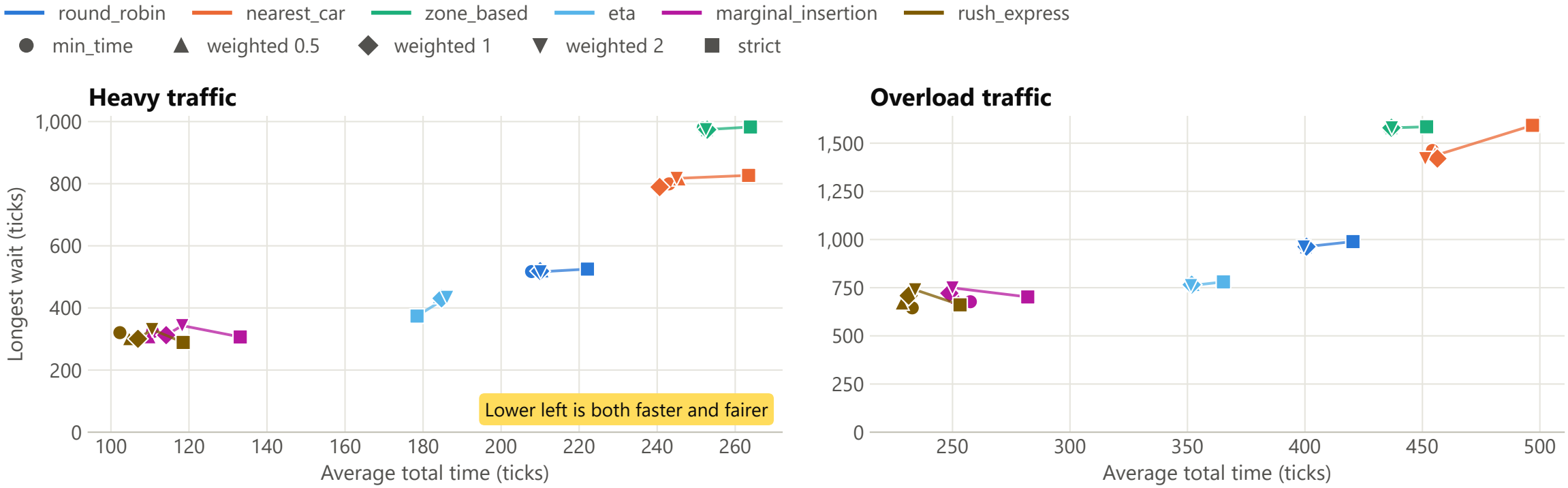
> **Algorithm:** `marginal_insertion` with `min_time`, the default guard against no guard, averaged over the four patterns and five seeds.

- On the short building the guard costs far more, and at heavy traffic p95 gets worse too.
- The guard costs capacity: measured with it, a building's limit falls by up to 17%, so on 4 of 8 building and pattern pairs heavy traffic is past the limit.

# 4. Seat rules

When a car has fewer seats than waiting passengers, who boards? Three rules, on all six algorithms, each with the guard on.

# With the guard on, seat rules change little




> **Algorithms:** One line for each of the six on `3car-60floor-6cap`, with one point per seat rule, averaged over the four patterns and five seeds. `min_time` and `weighted` run the default guard, and `strict` needs none.

- `min_time` seats the shortest ride. `weighted` lets a long wait count, at weights 0.5 to 2. `strict` seats in request order and keeps seats for earlier passengers.
- Each line runs from `min_time` to `strict`. The choice of algorithm moves a result far more than the seat rule does.

# What each seat rule changes, on each algorithm

| | `weighted 0.5`, heavy | | `strict`, heavy | | `weighted 0.5`, overload | | `strict`, overload | |
|---|---|---|---|---|---|---|---|---|
| Change against `min_time` | Average | Longest | Average | Longest | Average | Longest | Average | Longest |
| `round_robin` | +1% | 0% | +7% | +2% | 0% | 0% | +5% | +3% |
| `nearest_car` | +1% | +2% | +8% | +3% | 0% | −1% | +9% | +9% |
| `zone_based` | 0% | −1% | +5% | 0% | 0% | 0% | +4% | 0% |
| `eta` | 0% | +2% | −4% | −13% | 0% | 0% | +4% | +2% |
| `marginal_insertion` | −1% | −5% | +20% | −5% | −2% | +3% | +9% | +4% |
| `rush_express` | +2% | −6% | +16% | −10% | −2% | +4% | +9% | +2% |

> **Algorithms:** One row for each of the six on `3car-60floor-6cap`, averaged over the four patterns and five seeds. `min_time` and `weighted` run the default guard, and `strict` needs none.

- `weighted` moves both measures by under 15% on every algorithm. At overload it *lengthens* the longest wait on `marginal_insertion` and `rush_express`, at every weight.
- `strict` adds average time on every algorithm but `eta` at heavy traffic, and at overload its longest wait is no shorter than `min_time`'s on any of the six. The guard already protects the passenger who keeps being skipped.

# What improves, at what cost

| | |
|---|---|
| **The building** | Height sets the ride, and more floors also lengthen the wait. More cars and seats cut the wait, most of all when traffic is heavy. |
| **The algorithm** | Matters little with one car, and more as a building gets busy. |
| **Express cars** | Most layouts help at `up_peak` by separating traffic, and mostly cost elsewhere. `rush_express` splits the cars only during a lobby rush. |
| **Guards** | Cut the longest wait and the most skips for one passenger, for more average time. Seat hold does most of it, and the cost is larger on a short building. |
| **Seat rules** | With the guard on, `weighted` changes little, and `strict` mostly adds average time. |

This states what the runs show. It doesn’t name a best option: that depends on how much an average is worth against a worst case.

# Running the simulation

| Suite | Runs | Time, 8 workers | CPU time | One run: median | 95th percentile | Longest |
|---|---|---|---|---|---|---|
| Main experiment | 10,080 | 50 min | 6.5 h | 0.17 s | 8.5 s | 101 s |
| Eight buildings | 6,000 | 9 min | 1.1 h | 0.07 s | 1.8 s | 91 s |
| Express layouts | 4,800 | 13 min | 1.6 h | 0.11 s | 5.2 s | 70 s |
| **All** | **20,880** | **71 min** | **9.2 h** | | | |

**Inputs:** each building gets 4 patterns × 4 traffic levels × 5 seeds = 80 input files.

**Main experiment:** 3 buildings × 80 = 240 inputs. Stage 1 is 6 algorithms × 5 seat rules × 240 = 7,200. Stage 2 is 9 guard settings × 2 seat rules × 180 inputs (3 levels) = 3,240. 360 runs are in both, so 7,200 + 3,240 − 360 = **10,080**.

**Eight buildings:** 5 algorithms × 80 inputs × (8 buildings at their own limit + 7 on the reference's traffic) = **6,000**. The reference building's own runs are that traffic, so it counts once.

**Express layouts:** 6 algorithms × 80 inputs × (4 buildings at their own limit + 6 layouts on the passengers of the building they change) = **4,800**.

> **Measured on:** a Windows 11 laptop, AMD Ryzen 9 5900HX (8 cores, 16 threads), 31 GB of RAM, Python 3.15.2, nothing else running, with 8 runs at once. These figures describe the simulator, not how quickly the elevators serve passengers.

### Most runs take under a second

The slowest are all `eta`, `marginal_insertion` or `rush_express`, which simulate every car forward for each request, above all on the 10-floor building in overload.

### Memory is small

One run needs 21 to 23 MB, almost all Python itself. A suite of 8 workers stays under 300 MB, and drawing the charts peaks at 238 MB.

### Repeatable

The same input gives byte-identical files. Saved runs, 182 MB for the main experiment, let an interrupted suite carry on.

# Trade-offs and limits

| Choice | What it gives | What it costs |
|---|---|---|
| **Seat rules** | `min_time` favours the average, `strict` request order, and `weighted` sits between | `strict` adds average time, and with the guard on neither rule shortens the worst wait much |
| **The default guard, S = 2 with seat hold** | Roughly halves the worst wait | More average time, and up to 17% of a building's capacity |
| **Load levels measured with no guard, then frozen** | Every study uses identical passengers | With the guard on, heavy traffic is past the limit on 4 of 8 pairs |
| **Two kinds of load** | A building's own limit keeps every building equally busy, and shared traffic isolates the building | Shared traffic is gentle on big buildings |
| **Exact forecasts** | Car choices that weigh every passenger exactly | Up to 101 s for one run in overload |
| **Present, never rank** | No hidden judgement in the software | The conclusion is the presenter's to write |
| **No time guarantee** | Every request is served, and none refused | Guards limit skips, not time, and in overload queues grow |

Yellow means "look here", never better or worse. Final results. The default guard is on unless a slide says otherwise.

# How it was built and tested

### Standard library only

Python 3.15 and no other package to run it. Only the charts use `matplotlib`.

### Deterministic

The same input gives byte-identical files, and saved runs let an interrupted suite carry on.

### One movement model

Forecasts reuse the real boarding code, and over a million forecast decisions were checked against fresh ones.

### 686 tests

Hand-worked examples, property tests on random buildings and golden outputs. Separate scripts re-checked every passenger trip.

### Our adaptations

Seat hold, skip counts, shortest-ride-first boarding, seat reservation across floors and `rush_express` have no published elevator source.

### Tall buildings

60 floors is taller than any building in the published studies read, so there’s nothing to compare against.

# Conclusion

`marginal_insertion` looks like the strongest all-round algorithm. Unlike `eta`, it counts the delay to passengers already assigned. On every building with more than one car, it had the lowest average, p95 wait and longest wait at light and medium traffic. At heavy traffic its average was 35–65% below the next algorithm's, except at `up_peak`, where `zone_based` often did better.

| | |
|---|---|
| **Express cars, at the morning rush** | Against `marginal_insertion`, `rush_express` cut the average at `up_peak` by 27% at heavy traffic and 36% in overload, and changed nothing at other times. Fixed layouts helped at `up_peak` by up to 30%, but every one cost time under `uniform`. |
| **The guard, for fairness** | Cut the longest wait by 47% at heavy traffic, for 10% more average time. The most skips for one passenger fell from 6.5 to 2.5. |
| **Seat behaviour, with no guard** | `weighted` at 1 cut the longest wait by 39% for 9% more average time at heavy traffic, and `strict` by 49% for 32%. With the guard on, they add little. |

> **Algorithms:** The first point compares the five of the building study, each building at its own limit. The rest are `marginal_insertion`, and `rush_express` against it, on `3car-60floor-6cap`, averaged over five seeds and over the four patterns unless one is named. All with `min_time` and the default guard unless stated.

> **This is not academic research.** Stops take no time, the traffic is simulated, there are five seeds and no significance tests, several rules are our own adaptations, and nothing published covers 60 floors. The results compare options within this model, not how a real building will perform.

# Reference

The code, tests and README, the results with their charts, statistics and logs, and this presentation:

https://github.com/klgarychu/destination-dispatch-elevator-simulation